portada.pdf

Teoría de la Computación y Lenguajes Formales: Sesión 10. Evolución del Motor a PDA y Seguimiento de Workflows

Alcance de la sesión 10

De DFA a PDA: ¿Por qué evoluciona el Motor?

Extensión de la Ontología en YAML

fondo1280x720.pdf

Implementación del Motor PDA en Python

Seguimiento a un Flujo de Trabajo (Tracking)

Conclusión

fondo1280x720.pdf

Evolución de la Arquitectura: De DFA Plano a PDA con Memoria Estructurada

El Límite del Motor DFA de la Sesión 7

¿Cómo operaba el motor DFA básico?

En el Hito 1 programamos un motor que evaluaba transiciones directas basadas únicamente en el estado actual y el evento de entrada:

$$\delta(\text{estado_actual}, \text{evento}) \longrightarrow \text{nuevo_estado}$$

fondo1280x720.pdf

La necesidad en Workflows Complejos

Cuando un flujo de trabajo incluye **sub-procesos anidados o fases jerárquicas**:

- El sistema debe **recordar el contexto del proceso padre** al ingresar a un sub-workflow.
- Debe garantizar que las fases abiertas se cierren en el **orden inverso estricto (LIFO)**.
- Debe validar que no queden tareas subordinadas pendientes antes de dar por completado el producto.

La Especificación del PDA: Extensión de la Ontología en YAML



Estructura del YAML para el PDA

Declaración de la 7-Tupla en YAML

Añadimos a la especificación los elementos de la memoria de pila:

```
recurso: "Centro_Mecanizado_CNC"
tipo: "Estacion_Con_Subprocesos"
estado_inicial: "OCIOSO"
simbolo_inicial_pila: "Z0"
alfabeto_pila: ["Z0", "CTX_DESBASTE", "CTX_ACABADO"]

estados_finales:
  exito: ["COMPLETADO"]
  falla: ["BLOQUEO_CONTEXTO"]
```

Definición de Transiciones con Pila ($\delta : Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^*$)

```
transiciones_pda:
 # 1. PUSH: Iniciar sub-proceso de Desbaste (apila CTX_DESBASTE sobre Z0)
 - de: "OCIOSO"
   evento: "iniciar_desbaste"
   tope_pila: "Z0"
   a: "EN_DESBASTE"
   apilar: ["CTX_DESBASTE", "Z0"]

 # 2. POP: Finalizar sub-proceso de Desbaste (desapila CTX_DESBASTE)
 - de: "EN_DESBASTE"
   evento: "fin_desbaste"
   tope_pila: "CTX_DESBASTE"
   a: "DESBASTE_LISTO"
   apilar: []  # Cadena vacia (POP)
```

fondo1280x720.pdf

Implementación del Motor en Python: La Clase PDA y la Pila Física

La Clase PDA: Estructura de Datos

Inicialización con Pila LIFO

```
class PDA:
    def __init__(self, states, alphabet, stack_alphabet, transitions, q0, z0, final_states):
        self.states = states
        self.alphabet = alphabet
        self.stack_alphabet = stack_alphabet
        self.transitions = transitions  # Clave: (estado, evento, tope_pila) -> (nuevo_estado, push_list)
        self.q0 = q0
        self.z0 = z0
        self.final_states = final_states

        self.current_state = q0
        self.stack = [z0]  # Pila inicializada con el simbolo de fondo
        self.history = []

    def get_stack_top(self):
        return self.stack[-1] if self.stack else None
```

Lógica de Transición con Pila en Tiempo Real

Método de Evolución $\delta(q, e, Z)$

```
    def transition(self, event):
        if event not in self.alphabet:
            raise ValueError(f"[Error-Alfabeto]-Evento-'{event}'-no-reconocido-en-el-catalogo.")

        stack_top = self.get_stack_top()
        if stack_top is None:
            raise RuntimeError("[Error-Fatal]-Pila-vacia.-Bloqueo-de-ejecucion.")

        key = (self.current_state, event, stack_top)

        if key not in self.transitions:
            raise ValueError(
                f"[Error-de-Transicion]-No-existe-regla-para-({self.current_state},-'{event}',-tope={stack_top}).-"
                f"El-contexto-activo-no-permite-este-evento."
            )

        new_state, push_symbols = self.transitions[key]

        # 1. POP del simbolo actual del tope
        popped = self.stack.pop()

        # 2. PUSH de los nuevos simbolos (en orden inverso para que push_symbols[0] quede arriba)
        for sym in reversed(push_symbols):
            self.stack.append(sym)

        prev_state = self.current_state
        self.current_state = new_state
        self.history.append((event, prev_state, new_state, self.stack.copy()))

        print(f"->-Evento-'{event}':-{prev_state}-->-{new_state}-|-Pila:-{self.stack}")
```

Seguimiento de Workflows en Tiempo Real: Detección de Anomalías y Contexto

fondo1280x720.pdf

La Arquitectura de Seguimiento: Sensores y Gemelo Digital

El Sensor como Puente entre la Realidad y el Modelo

Para que el motor pueda realizar el seguimiento en tiempo real, los hechos del mundo físico u operacional deben convertirse en eventos discretos:

- **Sensor Automático (Físico / IoT):** Final de carrera, sensor fotoeléctrico, lector RFID/código de barras, señal de PLC.
- **Sensor Humano (Operador / Analista):** El operario que observa el estado de la pieza o el documento e introduce el evento en una terminal HMI/consola (`input()`).

f

El Motor como Gemelo Digital (*Digital Twin*)

El motor PDA mantiene en memoria el **Gemelo Digital** del proceso. Cada vez que un sensor emite un evento $e \in \Sigma$:

$$\text{Mundo Real} \xrightarrow{\text{Hecho Físico}} \text{Sensor/HMI} \xrightarrow{\text{Evento } e} \text{Motor PDA (Gemelo en RAM)}$$

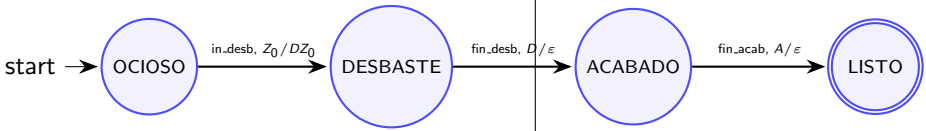
El gemelo evoluciona su estado y su pila. Si el evento contradice las reglas del proceso o el contexto activo, el gemelo **detecta la discrepancia y emite una alarma**.

Caso de Estudio: Mecanizado con Sub-procesos Anidados

Jerarquía del Flujo de Trabajo

El producto debe pasar obligatoriamente por dos sub-procesos jerárquicos:

1. **Sub-proceso 1: Desbaste Grueso** (Apertura → PUSH `CTX_DESBASTE`, Cierre → POP).
2. **Sub-proceso 2: Acabado Fino** (Apertura → PUSH `CTX_ACABADO`, Cierre → POP).



Monitoreo y Detección de Errores por el Motor

¿Cómo detecta el motor las anomalías operacionales?

- **Caso 1: Ejecución Normal Balanceada**

$$\text{in_desb} \rightarrow \text{fin_desb} \rightarrow \text{in_acab} \rightarrow \text{fin_acab}$$

  La pila inicia en $[Z_0]$, crece en cada sub-fase, se vacía a $[Z_0]$ y llega al estado marcado $\Rightarrow$ **ACEPTADO**.

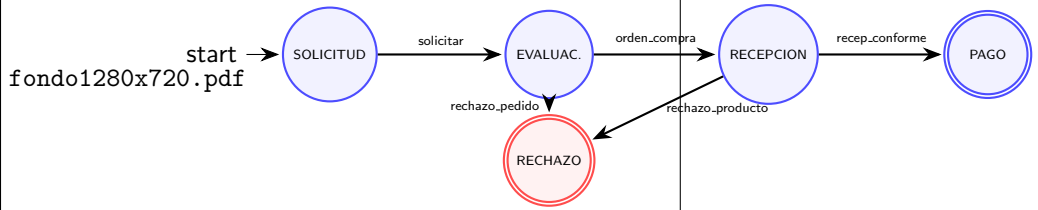
- **Caso 2: Intento de Salto de Fase (Violación de Contexto)** Si el operario intenta enviar `fin_acab` estando en la fase de desbaste: el tope de la pila es `CTX_DESBASTE`, por lo que la regla de acabado es rechazada de inmediato $\Rightarrow$ **ANOMALÍA DETECTADA**.

- **Caso 3: Tarea Inconclusa al Despachar** Si se envía el evento `despachar` mientras la pila contiene $[Z_0, \text{CTX_ACABADO}]$, el motor rechaza la finalización porque la pila no regresó a su símbolo base $\Rightarrow$ **PRODUCTO BLOQUEADO**.

Caso de Negocio: Proceso de Compras entre 4 Dependencias

El Gemelo Digital en Procesos Administrativos (ERP / BPMN)

Un pedido de compra transita por 4 dependencias (Solicitante → Compras → Almacén → Finanzas). Los analistas actúan como "sensores humanos" que registran eventos en el sistema:



Bifurcaciones que Cortan el Flujo

Los eventos de rechazo (`rechazo_pedido` o `rechazo_producto`) desvían inmediatamente el token hacia el estado de falla (`RECHAZO`), cortando el flujo e impidiendo que se genere la orden de pago.

Conclusión de la Sesión 10

Logros del Motor PDA

- **Control de Contexto Dinámico:** La Pila permite al motor saber exactamente en qué nivel de sub-proceso se encuentra el producto.
- **Validación de Balances:** Garantiza que ninguna fase de trabajo quede abierta o se cierre fuera de orden.
- **Trazabilidad Total:** El historial de ejecuciones registra no solo los estados, sino la evolución de la memoria de contexto en cada evento.

f

Próximo Bloque (Semanas 10-12): Concurrencia y Redes de Petri

Ya dominamos el control de un recurso con sub-procesos anidados. En la próxima clase daremos el salto a la **concurrencia**: múltiples recursos comunicándose mediante **paso de mensajes ($!a/?a$) y Redes de Petri**.

¡Muchas gracias!

salida.pdf